Lab 0: Booting 实验报告

潘孝圆　24300240128　2026 年 10 月 16 日

本次修改文件为 `src/main.c`。实际执行顺序是：CPU 0 清零 BSS，唤醒其余核心并初始化打印服务，输出自己的问候，再放行其余核心打印。

任务 1：CPU 0 输出问候

思路与实现：在 CPU 0 分支中，等 `uart_init()` 和 `printk_init()` 完成后，使用 `printk()` 输出 `Hello, world! (Core 0)` 并换行。打印位于放行其他核心之前，因此 Core 0 首先出现。

交流摘要：我问“ok 写完了，接下来要干啥”。模型先解释 `printk()` 类似 C 中的 `printf()`，再带我找到对应 TODO，说明为什么要等打印服务初始化、为什么字符串末尾需要换行。我根据讲解手动补上这一处输出。

任务 2：其余核心输出各自编号

思路与实现：其他核心先等待 `boot_secondary_cpus` 变为 `true`，经过原有 `arch_fence()` 后，再使用 `cpuid()` 取得本核编号，将其转换为 `int` 并配合 `%d` 输出。三个核心并行执行，获取打印锁的顺序不固定，所以后三行可以交换顺序。

交流摘要：在同一轮讲解中，模型接着说明“当前核心编号”如何填入格式字符串，区分固定的 Core 0 输出与随核心变化的输出。我参照接口示例填写另一个 TODO，再运行并反馈终端结果，检查各核心是否都出现一次。

任务 3：初始化前清零 BSS

思路与实现：BSS 中本实验使用的未显式初始化全局、静态对象需要零初值，启动时应保证这块内存已清零。复用链接脚本已有的 `edata`、`end`，用 `extern char edata[], end[];` 引用地址，再在 `smp_init()` 之前调用：

```
memset(edata, 0, (usize)end - (usize)edata);
```

清零范围为 `[edata, end)`，第三个参数是两地址相差的字节数；本次范围覆盖 BSS，`linker.ld` 保持原样。

交流摘要：我先问“怎么找起始地址和结束边界啊”，又追问“为啥这么做啊”。模型用地址标记和逐字节填零的例子，解释 `extern`、`memset` 三个参数及边界不包含在范围内的含义。讨论后采用已有符号，我手动完成声明和清零调用。

运行结果与工具使用

执行 `cmake --build build --target qemu` 后，最终验证输出顺序为 Core 0、Core 2、Core 1、Core 3，四核各出现一次。原先字符串中的小写 `core` 经检查改为 `Core`。

本次使用 Codex 辅助学习，模型可见标识为 GPT-6。以上对话为教学过程摘要：我不断追问原理和操作位置，模型逐步讲解并提供关键调用示例，我据此手动修改并运行验证；报告也由该工具辅助整理与 LaTeX 排版。